

Fine Tuning

We'll build one complete pipeline from **problem definition** → **baseline** → **data** → **cleaning** → **LoRA** → **QLoRA** → **experiments** → **evaluation** → **error analysis** → **conclusions**. At every stage, we'll first establish *why* we are doing something, then implement it.

For the model, HuggingFaceTB/SmolLM2-135M-Instruct is a very good teaching choice: it is small enough to iterate quickly, already instruction-tuned, and Hugging Face provides Transformers support and a chat template. The model card says the 135M model was pretrained on about 2T tokens and subsequently instruction-tuned and DPO-trained. ([Hugging Face](#))

Our project

Rather than fine-tuning on an arbitrary dataset, let's create a problem where we can **clearly measure whether fine-tuning actually helped**.



Problem: Technical Support Intent + Structured Response

We'll teach the model to behave like a small **technical-support classifier/assistant**.

Given a user's support request, the model should produce a predictable structured response:

INTENT: password_reset

PRIORITY: medium

ACTION: Provide password reset instructions

For example:

Input

I forgot my password and can't log into my account.

Expected output

INTENT: password_reset

PRIORITY: medium

ACTION: Provide password reset instructions

Another:

Input

Someone got access to my account and changed the email address.

Expected output

INTENT: account_compromise

PRIORITY: high

ACTION: Secure the account and initiate account recovery

This is deliberately somewhat artificial.

That's a feature, not a bug.

We want a task where we can answer:

Did fine-tuning actually change the model's behavior in the way we wanted?

rather than vaguely saying:

"The responses seem better."

The course roadmap

We'll go through this as a sequence of labs.

Lab 0 — Environment

We'll establish:

Python

PyTorch

Transformers

Datasets

PEFT

TRL

bitsandbytes

Accelerate

and check:

CPU / GPU

CUDA

VRAM

package versions

Lab 1 — Define the problem

We'll define:

Task

Input contract

Output contract

Labels

Success criteria

Failure criteria

Evaluation set

Most importantly, we'll distinguish:

training objective

from

business/task objective

and from

evaluation metric.

That's one of the most important concepts in practical fine-tuning.

Lab 2 — Establish the baseline

We'll load:

[HuggingFaceTB/SmolLM2-135M-Instruct](#)

and inspect:

model

tokenizer

config

parameters

trainable parameters

context length

chat template

Then we'll manually ask it questions.

For example:

User:

I forgot my password and can't access my account.

Assistant:

???

We'll save the actual outputs.

This becomes our **pre-fine-tuning baseline**.

The model card explicitly shows using `apply_chat_template()` for this model, so we'll respect the model's native conversational format rather than inventing our own prompt serialization. ([Hugging Face](#))

We'll also deliberately probe the baseline with examples it hasn't seen.

Lab 3 — Define the output contract

This is extremely important.

We aren't going to say:

"Generate a good answer."

We'll define something measurable.

For example:

INTENT: <one allowed intent>

PRIORITY: <low|medium|high>

ACTION: <one concise action>

Our allowed intents might initially be:

password_reset

billing_issue

account_compromise

refund_request

technical_problem

feature_request

account_closure

Now we can automatically check:

output_has_valid_intent()

output_has_valid_priority()

output_has_valid_action()

output_matches_expected_intent()

That gives us a bridge between:

LLM generation

and

traditional software testing.

Lab 4 — Build the dataset

We'll start small.

Something like:

500–2,000 examples

is plenty for the learning exercise.

We'll deliberately construct examples covering:

Easy examples

I forgot my password.

Paraphrases

I can't remember the password for my account.

Informal language

ugh I can't get into my account, forgot my pass

Typos

I forgot my password

Ambiguous examples

I can't get into my account anymore.

Adversarial-ish examples

My password works but someone changed my recovery email.

The last categories are important because they let us test whether the model learned **the task** rather than simply memorizing phrases.

Lab 5 — Dataset schema

We'll settle on a canonical representation.

For example:

```
{
  "id": "support_000001",
  "messages": [
    {
      "role": "user",
      "content": "I forgot my password and can't log in."
    },
    {
      "role": "assistant",
      "content": "INTENT: password_reset\nPRIORITY: medium\nACTION: Provide password reset instructions"
    }
  ],
  "intent": "password_reset",
  "priority": "medium"
}
```

Notice something important here:

We can retain the **structured metadata**:

intent

priority

even though the actual SFT target is natural language.

That gives us powerful evaluation possibilities later.

Lab 6 — Data cleaning & normalization

We'll build actual cleaning code rather than manually editing the dataset.

We'll investigate:

Duplicate detection

exact duplicates

near duplicates

Invalid labels

priority = "urgent"

when only:

low

medium

high

are allowed.

Empty examples

user = ""

assistant = "..."

Bad formatting

INTENT:password_reset

PRIORITY : medium

Excessively long examples

Leakage

For example:

User:

What intent should this example have?

Assistant:

password_reset

which may accidentally make the task trivial.

Lab 7 — Train/validation/test split

We'll establish this **before training**.

For example:

70% train

15% validation

15% test

But we'll also discuss why a random split can be misleading.

Suppose we have:

"I forgot my password"

"I can't remember my password"

"My password slipped my mind"

If all three end up in different splits, our test set isn't really independent.

So we'll eventually experiment with **semantic/group-aware splitting**.

This is where the class becomes much more realistic than a typical fine-tuning notebook.

Lab 8 — Tokenization analysis

Before training, we'll inspect what the tokenizer actually sees.

For example:

```
tokens = tokenizer(text)
```

We'll examine:

characters

words

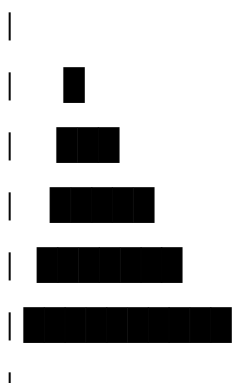
tokens

token IDs

sequence lengths

We'll plot the distribution:

sequence length



This will help us understand:

- padding
- truncation
- context length

- memory consumption
 - batch size
 - why sequence length matters so much
-

Lab 9 — First training run: LoRA

Now we finally fine-tune.

We'll use **PEFT/LoRA**, not full fine-tuning.

Conceptually, instead of changing:

W

we learn a low-rank update:

$$W' = W + \Delta W$$

$$\Delta W \approx B A$$

where:

$$A \in \mathbb{R}^{(r \times d)}$$

$$B \in \mathbb{R}^{(d \times r)}$$

and:

$$r \ll d$$

So instead of learning every parameter in W , we learn a much smaller number of parameters.

Hugging Face's current PEFT/TRL documentation describes this setup as freezing the base model while training the additional adapter parameters. ([Hugging Face](#))

We'll inspect this ourselves:

```
model.print_trainable_parameters()
```

and actually calculate:

total parameters

trainable parameters

percentage trainable

rather than treating LoRA as magic.

Lab 10 — Understand the LoRA configuration

We'll experiment with:

```
LoraConfig(
```

```
    r=...
```

```
    lora_alpha=...
```

```
lora_dropout=...
target_modules=...
)
```

and understand each parameter.

Especially:

r

Controls adapter rank.

We'll test something like:

r = 4

r = 8

r = 16

r = 32

and observe:

trainable parameters

training behavior

validation loss

test performance

lora_alpha

We'll understand why:

α / r

matters.

target_modules

This is especially important.

We'll inspect the actual architecture and determine **which linear layers we are adapting** rather than blindly copying:

```
target_modules=["q_proj", "v_proj"]
```

from some unrelated model.

Lab 11 — First controlled experiment

We'll establish a reproducible baseline experiment.

Something like:

Model:

SmolLM2-135M-Instruct

Method:

LoRA

Dataset:

version 1

r:

8

alpha:

16

dropout:

0.05

learning_rate:

...

epochs:

...

seed:

42

We'll save this as an experiment configuration.

Then:

train

↓

validate

↓

save adapter

↓

evaluate test set

↓

record metrics

The point is that **every run becomes reproducible.**

Lab 12 — Metrics

We'll use several layers of evaluation.

Training metrics

training loss

validation loss

learning rate

gradient norm

Task metrics

For our structured task:

intent accuracy

priority accuracy

format validity

exact-match accuracy

Potentially:

macro F1

as well.

Generation metrics

We'll also inspect:

output length

invalid outputs

missing fields

hallucinated fields

And then manually inspect examples.

Because a single metric isn't enough.

Lab 13 — The important experiment: does it generalize?

We'll deliberately create a **challenge test set**.

For example:

Training

I forgot my password.

Test

My credentials have completely slipped my mind.

If the model learned:

"forgot password" → password_reset

it may struggle.

If it learned the underlying intent:

unable to authenticate because credential is forgotten

it should succeed.

This distinction is central to fine-tuning.

Lab 14 — Overfitting experiment

We'll intentionally train too much.

For example:

1 epoch

3 epochs

10 epochs

20 epochs

Then observe:

train loss ↓↓↓

validation loss ↓ then ↑

test accuracy ↓

if we overfit.

This will make the concept of overfitting concrete rather than theoretical.

Lab 15 — LoRA hyperparameter experiment

We'll create an experiment matrix.

For example:

Experiment	r	α	dropout	LR
A	4	8	0.05	2e-4
B	8	16	0.05	2e-4
C	16	32	0.05	2e-4
D	32	64	0.05	2e-4
E	8	16	0.10	2e-4

Then compare them systematically.

Not:

"I tried $r=16$ and it seemed better."

But:

"Under the same seed, dataset, optimizer and training budget, increasing rank from 4 $\rightarrow$ 16 improved intent accuracy by X while increasing trainable parameters by Y."

That's the kind of reasoning we want.

Lab 16 — QLoRA

Then we'll do the same thing using **QLoRA**.

The key distinction:

LoRA

FP16/BF16 base model

↓

frozen

+

LoRA adapters

↓

train

QLoRA

4-bit quantized base model

↓

frozen

+

LoRA adapters

↓

train

Hugging Face's TRL documentation describes QLoRA as combining 4-bit quantization with LoRA, with the quantized base weights frozen and the adapters trained separately. ([Hugging Face](#))

We'll actually inspect the model:

```
print(model.dtype)
```

and the quantization configuration.

We'll learn why:

4-bit weights

≠

4-bit training

The distinction matters.

Lab 17 — QLoRA configuration

We'll introduce:

```
BitsAndBytesConfig(  
    load_in_4bit=True,  
    bnb_4bit_quant_type="nf4",  
    bnb_4bit_compute_dtype=...  
)
```

and understand what each setting means.

We'll also discuss:

NF4

FP4

FP16

BF16

compute dtype

storage dtype

rather than simply copying a configuration.

Lab 18 — LoRA vs QLoRA experiment

We'll compare:

LoRA	QLoRA
------	-------

Base precision

VRAM

Trainable params

Training time

Loss

Accuracy

Output quality

Adapter size

For a 135M model, QLoRA won't necessarily give us a dramatic practical advantage.

And that's actually an **important lesson**.

QLoRA's major value becomes increasingly obvious with much larger models.

We're using a tiny model so that we can understand the mechanism without needing expensive hardware.

Lab 19 — Adapter inspection

We'll inspect what was actually saved.

Instead of saving:

135M model

we should be able to save an adapter containing only the learned LoRA parameters.

We'll examine:

adapter_config.json

adapter_model.safetensors

and understand:

base model

+

adapter

=

fine-tuned model

This is one of the most useful practical concepts in PEFT.

Lab 20 — Merge vs adapter inference

We'll compare:

base + adapter

against:

merged model

and understand when you would choose each.

Lab 21 — Error analysis

This is where we'll spend serious time.

We'll create a table like:

Input	Expected	Predicted	Error
Forgot password	password_reset	password_reset	—
Recovery email changed account	account_compromise	password_reset	semantic confusion
Can't login	password_reset	technical_problem	ambiguity
Charged twice	billing_issue	refund_request	label boundary

Then we'll ask:

Why did the model fail?

Possible causes:

bad data

ambiguous labels

insufficient examples

class imbalance

overfitting

prompt formatting

model capacity

tokenization

training configuration

This is where we'll learn that **fine-tuning isn't just optimization**.

A large part of successful fine-tuning is **problem/data engineering**.

Lab 22 — Data improvement experiment

We'll intentionally improve the dataset.

For example:

Version 1

"I forgot my password"

Version 2

Add paraphrases:

"I can't remember my password"

"I lost access because I forgot my password"

"My password isn't coming to mind"

Version 3

Add difficult negatives:

"My password works, but someone changed my recovery email"

Then retrain.

We'll be able to answer:

Did the additional data help more than increasing LoRA rank?

That's a much more valuable engineering question than simply tuning hyperparameters.

Lab 23 — Ablation studies

Eventually we'll have experiments like:

Base model



Base + LoRA

Base + LoRA



Base + QLoRA

Base + LoRA



better dataset

Base + LoRA



higher rank

Base + LoRA



more epochs

Then we'll determine which intervention actually mattered.

Lab 24 — Build an evaluation harness

By the end, we won't manually run:

```
model.generate(...)
```

for every example.

We'll build:

```
evaluate.py
```

which does:

```
load model
```



```
load evaluation dataset
```



```
generate outputs
```



parse structured output



calculate metrics



save predictions



produce report

Something like:

Evaluation Report

=====

Model: SmolLM2-135M-Instruct

Adapter: experiment_007

Examples: 300

Format validity: 97.3%

Intent accuracy: 91.7%

Priority accuracy: 88.4%

Exact match: 84.2%

Failed examples: 47

Now we have an actual evaluation pipeline.

Lab 25 — Reproducibility

We'll make the project reproducible with:

seed

dataset version

model revision

library versions

training configuration

evaluation configuration

experiment ID

We'll eventually have something like:

experiments/

|

├─ exp_001_baseline/

├─ exp_002_lora_r4/

├─ exp_003_lora_r8/

├─ exp_004_lora_r16/

├─ exp_005_lora_r32/

├─ exp_006_qlora_r8/

└─ exp_007_better_dataset/

Each experiment contains:

config.json

metrics.json

predictions.jsonl

training_log.json

adapter/

That's how we'll turn a notebook experiment into something resembling an actual ML workflow.

One important choice

I recommend **not starting with a huge dataset**.

Our first dataset can be intentionally tiny:

~300 training examples

~75 validation examples

~150 test examples

Then we'll expand it.

Why?

Because when something goes wrong, we can actually inspect it.

If we start with 100,000 examples, we'll have no idea whether a problem came from:

data

tokenization

formatting

trainer

LoRA configuration

evaluation

Small → understandable → controlled → scale.

The stack

Our core stack will be:

Python

|

|— PyTorch

|

|— Transformers

|

|— Datasets

|

|— PEFT

| └─ LoRA

|

|— TRL

| └─ SFTTrainer

|

|— bitsandbytes

| └─ QLoRA / 4-bit

|

└─ Accelerate

TRL's current SFTTrainer directly supports PEFT configurations, and its documentation also exposes quantization configuration for QLoRA. ([Hugging Face](#))

How we'll conduct each lesson

I suggest a strict rhythm:

1. Concept

I'll explain the minimum theory necessary.

2. Inspect

We'll inspect what the model/data actually looks like.

3. Implement

We'll write the code.

4. Run

We'll execute it.

5. Inspect the result

We'll look at actual outputs/metrics.

6. Break it

Where useful, we'll deliberately introduce a mistake.

7. Explain why

Then we'll fix it.

That last part is important.

For example, instead of simply saying:

"Use target_modules=['q_proj', 'v_proj']."

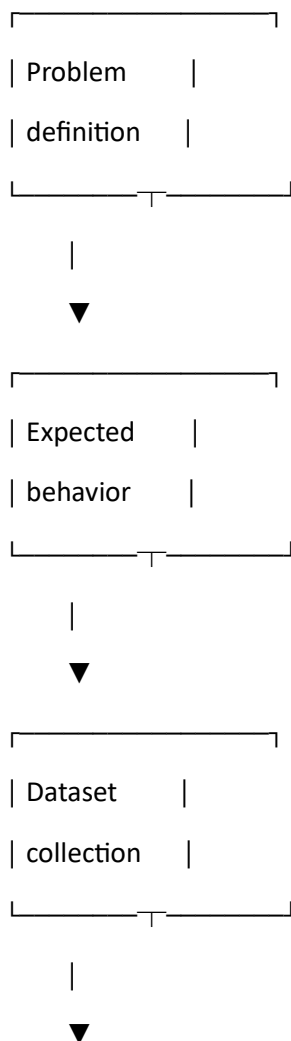
we'll inspect the model's modules and ask:

"What happens if we target the wrong modules?"

That is how the implementation becomes **crystal clear**.

Our final mental model

By the end, I want the entire workflow to be something you can visualize as:



Cleaning &
normalization

|



Train / Val /
Test

|



Base model
baseline

|



LoRA
experiment

|



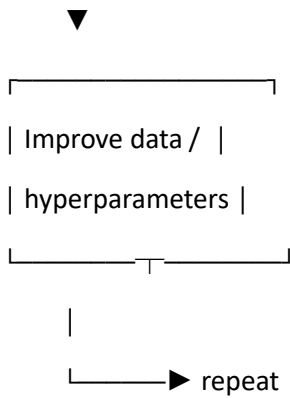
QLoRA
experiment

|



Evaluation & error analysis

|



And, crucially:

Fine-tuning

≠

"make loss go down"

Our actual objective is:

Problem



Desired behavior



Measurable evaluation



Data



Training



Evidence



Decision

That's the mindset we'll use throughout.

Let's start: Lab 0 → Lab 1

Our first hands-on exercise will be to set up the environment and then inspect SmolLM2-135M-Instruct before writing a single training line.

We'll start by answering four concrete questions:

1. **What exactly is this model?**
2. **What architecture does it use?**
3. **How many parameters are there?**
4. **What does one real prompt → token IDs → generation actually look like?**

Then we'll define our dataset schema and create our first examples.

I won't jump ahead to SFTTrainer yet. **We first establish the baseline and understand the model we're about to modify.**